GSoC21 W1: LFortran Kickoff

Rohit Goswami · 2021-06-12 · gsoc21 · fortran · ramblings

> *Charting paths towards concrete `lfortran` usage*

Background

As mentioned in an earlier post, I have had the immense pleasure of continuing development of the disruptive `lfortran` compiler under the aegis of the Fortran Lang organization, financed by the Google Summer of Code and mentored ably by Ondrej Certik. A rather interesting consequence of this is that we are strongly encouraged to write a precis of our activities each week. This is mine, given that the clock starting winding down (full timeline) last Monday.

I will workshop a few styles of getting maximal information into the post while ensuring the bulk of the technical details and other related notes makes its way into the appropriate channels, that is, into the `lfortran` project repository and documentation.

Logistics

- I met with Ondrej on Thursday, though our standing arrangement is for Tuesday
  - Meeting earlier in the week helps reduce chances of veering off course
  - Though an equitable distribution of work is assumed, weekends tend to be more conductive to bursts of effort
- We decided to release progress reports every Friday night for community oversight and approval
  - Slight discontinuity here, in that the weekend remains, but the idea is to take community suggestions per week as well
- We discussed the work completed by my fellow GSoC student developers over the community bonding period
- Discussed the current minimum viable project issue relating to SNAP [^fn:1], the SN (Discrete Ordinates) Application Proxy from LANL
  - Also looked into starting work on the issue relating to `dftatom` which forms the crux of my project
- Determined approaches towards the final goal
  - Discussed the possibility of making a first pass in the `C++` backend instead of the `LLVM` one
    - **Con:** Is not very fast
    - **Pro:** Is very easy
- Also discussed adding generic types, e.g. `type(*)` to **internal** routines for the compiler
  - Saves a lot of boilerplate code
  - Could start with a simple `fypp` based approach (similar to `fpm`)
  - Should be fine as long as users do not start using them

As per my initial project plan, the goal for the week (in progress) is the **Enumeration of language features used in the dftatoms project and cross correlating it with implemented features in LFortran**. To this, we decided that as the AST works well enough, it would be better to follow in the footsteps of the SNAP issue and determine the dependency tree, before working through it steadily.

Setting up LFortran and dftatom

One approach to doing this was naturally, to compile everything by hand, painstakingly using the logical dependency graph which comes with the repository. Note that for completion (and to add unnecessary gravitas to this post), we include **once** the setup details for `lfortran` and `dftatom` which have been included verbatim from the documentation [^fn:2].

LFortran

On Linux systems, I use `nix` (naturally). On a Mac, `nix` has unresolved issues upstream, so it is easier to use `micromamba` (here) or some other `anaconda` helper.

```
# Linux
git clone git@gitlab.com:lfortran/lfortran
cd lfortran
nix-shell ci/shell.nix
./build0.sh
./build1.sh
export PATH=$(pwd)/inst/bin/:$PATH
./test_lfortran_cmdline
./run_tests.py
# MacOS
micromamba create -p ./tmp llvmdev=11.0.1 bison=3.4 re2c python cmake make toml -c conda-forge
micromamba activate tmp
export PATH=$(pwd)/tmp/bin:$PATH
./build0.sh
cmake -DCMAKE_BUILD_TYPE=Debug -DWITH_LLVM=yes -DCMAKE_INSTALL_PREFIX=`pwd`/inst .
make -j$(nproc)
ctest
./run_tests.py
```

dftatom

Another post can go over why `dftatom` was chosen as a benchmark code, but to keep things short, it is well written, has a great `cython` wrapper, is fast, and is useful Čertík, Pask, and Vackář (2013). Rather than write a `nix` shell for it though, I opted to get started with the `anaconda` set of instructions to test with.

```
git clone git@github.com:certik/dftatom
cd dftatom
micromamba create -p ./tmp cython numpy python=3.8 pytest hypothesis -c conda-forge
cmake -DCMAKE_INSTALL_PREFIX=$CONDA_PREFIX -DCMAKE_PREFIX_PATH=$PREFIX -DWITH_PYTHON=yes .
make
make install
PYTHONPATH=. python examples/atom_U.py
```

Later in the week (over the weekend) I might add some quality of life commits like a `nix` environment.

Testing the AST

The logic behind the AST test is that if we can parse each file with `lfortran` and write it back out into the `f90` files which can then be compiled by `gfortran` and then subsequently pass our tests; then the AST is roughly alright.

Naturally this still strips comments and therefore `openmp` directives, of which `dftatom` thankfully has none.

Now it so happens that this is fairly trivial with `lfortran` and `bash`.

```
cd $DFTATM/src # src under dftatom repo
find . -name "*.f90" -exec lfortran fmt -i {} \;
# Rebuild and test with gfortran
```

This worked well. No glaring errors yet. Which brings us to the ASR representation [^fn:3].

Testing the ASR

To start testing the ASR with `lfortran -c fname.f90 -o fname.o` and variations thereof, a good starting point is to set up a dependency graph. A logical one is provided in the repository itself as seen in Fig. 2.

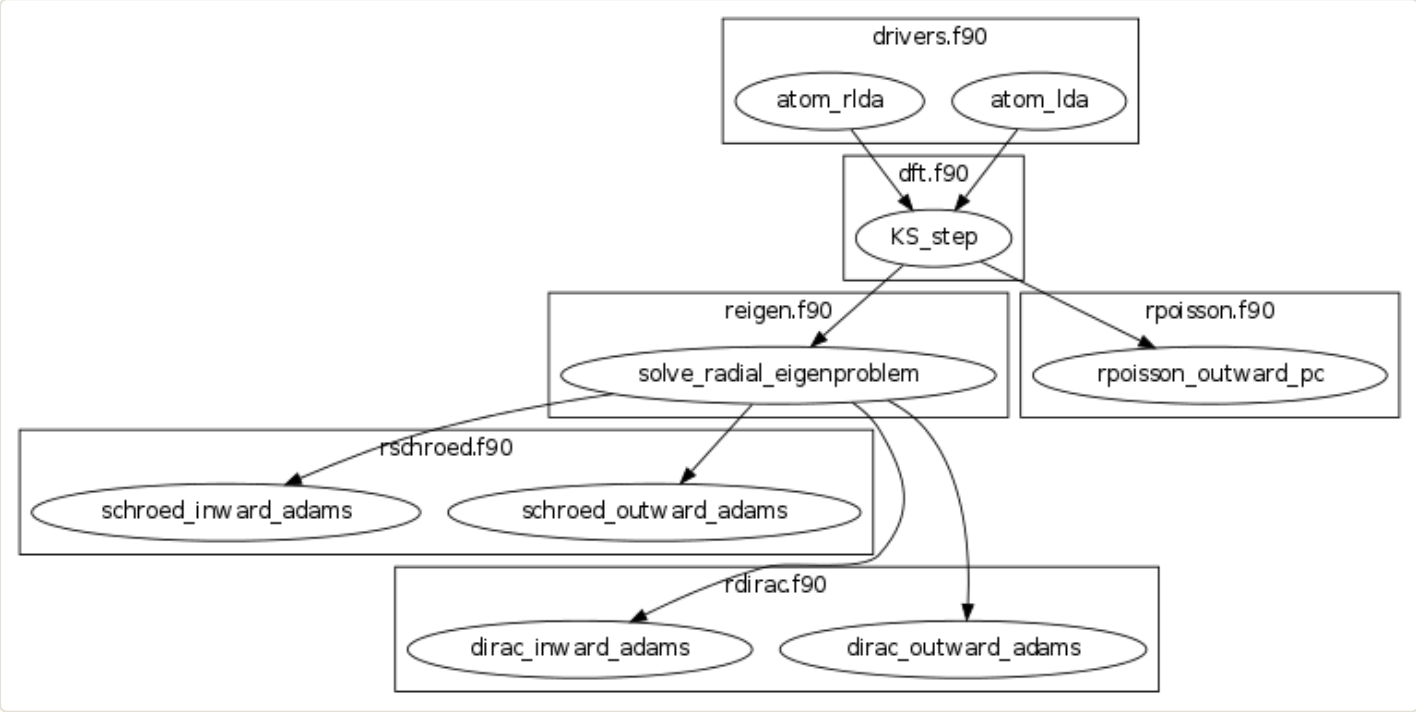

Figure 1: `dftatom` logical layout

However, this is not exactly the order in which things are built. Thankfully, the fantastic `fortdepend` tool from Peter Hill can speed things up considerably.

```
# in the conda environment
pip install fortdepend
cd $DFTATM/src
fortdepend -o Makefile.dep
```

This can and was then manually inspected and marshalled by order of dependencies into the issue. Here we will conclude with the visual details instead of dumping the entire dependency tree.

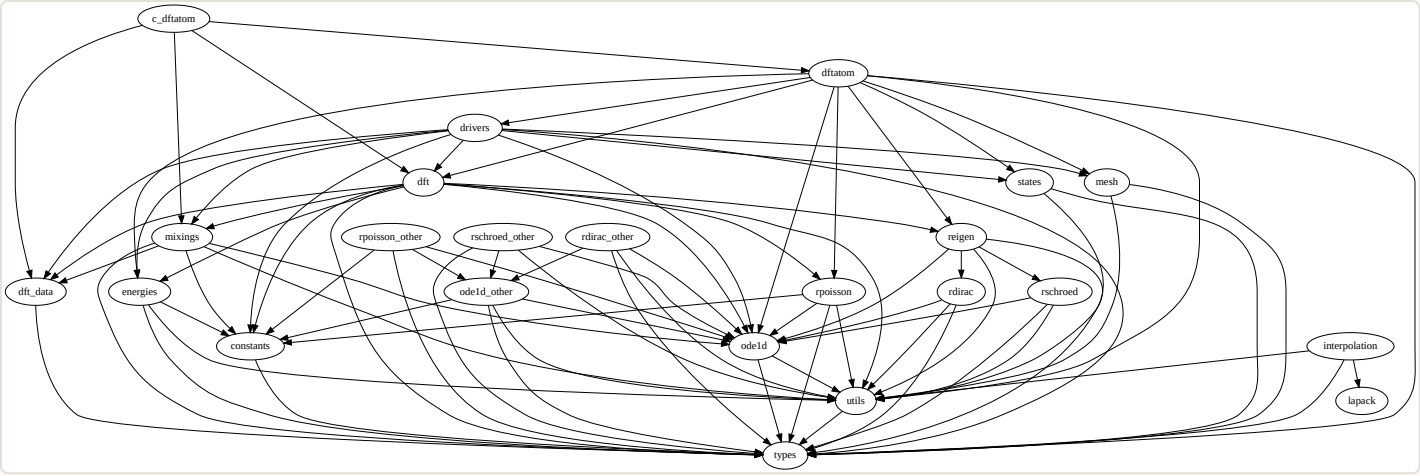

Figure 2: `dftatom` compilation dependency graph

Conclusions

That leads us to the end of this week's effectively mid-week update. For the remainder of the week, until the meeting with Ondrej I shall work through parts of the dependency graph and write some more about `dftatom` and the `lfortran` terminology which will be used for the remainder of the project.

References

Čertík, Ondřej, John E. Pask, and Jiří Vackář. 2013. "Dftatom: A Robust and General Schrödinger and Dirac Solver for Atomic Structure Calculations." *Computer Physics Communications* 184 (7): 1777--91. <https://doi.org/10.1016/j.cpc.2013.02.014>.

```
LAMMPS
```

```
earlier](https://gitlab.com/lfortran/lfortran/-/merge_requests/694)
and [fixed a
typo](https://gitlab.com/lfortran/lfortran/-/merge_requests/975) in
now
```

```
up a lot
```